

MAS Final Project: Configuration and Parameters (`config.py` + `config.yaml`) Complete Code Documentation

1. Scope

This chapter documents all configuration code and runtime parameters, covering:

- `config.py` constants and functions,
- YAML override loader behavior,
- parameter groups and their runtime effects,
- precedence rules between defaults, YAML, and CLI.

2. Module Purpose

`config.py` is the central configuration registry for the project. It provides:

- typed module-level defaults,
- optional YAML-based overrides,
- global constants consumed by `environment/planner/verification/refinement/CLI`.

`config.yaml` is the default external override file loaded at runtime.

3. Imports and Optional Dependency

Imports:

- `Path` from `pathlib`
- `logging`
- `List`, `Optional`, `Tuple` from `typing`

Optional import:

- `yaml` (`PyYAML`), guarded in `try/except`.

If import fails:

- `yaml = None`
- YAML override loading is skipped with warning.

Logger:

- `logger = logging.getLogger("warehouse.config")`

4. Visual Constants (Color Palette)

`config.py` defines multiple RGB tuples for rendering in `env.py`. Examples:

- base colors: WHITE, BLACK, GRAY, DARK_BLUE, TEAL, ORANGE, RED, GOAL_COLOR, GOLD, GREEN
- UI palette: BG_PURE, GRID_SUBTLE, ROBOT_PRIMARY, ROBOT_CARRYING, ROBOT_SHADOW
- shelf colors: SHELF_IDLE, SHELF_ACTIVE, SHELF_SHADOW
- goal colors: GOAL_PRIMARY, GOAL_SHADOW
- text/accent: TEXT_PRIMARY, TEXT_ON_DARK, ACCENT_GOLD

These are presentation parameters only; they do not affect logic or safety.

5. Environment Defaults

Defined constants:

- GRID_W: int = 16
- GRID_H: int = 16
- CELL_SIZE: int = 80
- NUM_AGENTS: int = 6
- NUM_SHELVES: int = 8
- GOALS: List[Tuple[int, int]] = [(7, 14), (8, 14)]

Usage:

- consumed primarily by WarehouseEnv constructor and render sizing.

6. Planning Defaults

Defined constants:

- ACTION_SIZE: int = 5
- PLAN_HORIZON: int = 30
- ASTAR_MAX_NODES: int = 3500
- IDLE_LIMIT: int = 4
- RESERVATION_WINDOW: int = 8
- UNPLANNED_HOLD_STEPS: int = 2
- ESCAPE_IDLE_STEPS: int = 6
- RENDER_FPS: int = 2

Primary consumers:

- pathfinding.py (CooperativePlanner tuning values)
- env.py (render_fps)
- main.py defaults (plan_horizon)

7. Verification and Refinement Defaults

Defined constants:

- MIN_SEPARATION: int = 1
- VERIFY_HORIZON: int = 30

- `VERIFY_TRIALS: int = 20`
- `REFINE_ITERATIONS: int = 2`
- `REFINE_MAX_CONSTRAINTS: int = 100`

Consumers:

- `verification.py`
- `main.py` verify-refine orchestration
- `refinement.py` cap inputs (via main args)

8. State Size Function

Function:

- `state_size() -> int`

Formula:

- $4 + \text{NUM_SHELVES} * 4 + (\text{NUM_AGENTS} - 1) * 4$

Meaning:

- 4 self features,
- 4 per configured shelf slot,
- 4 per other robot slot.

Exported constant:

- `STATE_SIZE: int = state_size()`

This value updates after YAML load to stay consistent with overridden agent/shelf counts.

9. YAML Loader (`load_from_yaml`)

Function:

- `load_from_yaml(path: Optional[str] = None) -> None`

Purpose:

- load optional overrides from YAML file and mutate module globals.

9.1 Globals Modified

The function updates:

- environment:
 - `GRID_W`, `GRID_H`, `CELL_SIZE`, `NUM_AGENTS`, `NUM_SHELVES`, `GOALS`
- planning:
 - `PLAN_HORIZON`, `ASTAR_MAX_NODES`, `IDLE_LIMIT`
 - `RESERVATION_WINDOW`, `UNPLANNED_HOLD_STEPS`, `ESCAPE_IDLE_STEPS`, `RENDER_FPS`

- verification:
 - `MIN_SEPARATION`, `VERIFY_HORIZON`, `VERIFY_TRIALS`
- refinement:
 - `REFINE_ITERATIONS`, `REFINE_MAX_CONSTRAINTS`
- derived:
 - `STATE_SIZE`

9.2 Path Resolution and Safety Guards

1. If `path` provided, use it; else default `config.yaml`.
2. If file does not exist:
 - debug log and return.
3. If `yaml` module unavailable:
 - warning log and return.
4. Parse YAML with `yaml.safe_load`.
5. If parsed root is not mapping (`dict`):
 - warning log and return.

9.3 Section Mapping Rules

Expected top-level YAML sections:

- `grid`
- `agents`
- `planning`
- `render`
- `verification`
- `refinement`

Each known key is converted using `int(...)` (or tuple conversion for goals).

Unknown keys are ignored.

9.4 Goals Parsing

If `agents.goals` is a list:

- converts each item to tuple:
 - `GOALS = [tuple(goal) for goal in agents["goals"]]`

Expected shape is list of 2-element numeric lists.

9.5 Post-Load Recompute

After all assignments:

- recomputes `STATE_SIZE = state_size()`
- logs successful load info with path.

10. Parameter Groups and Runtime Effects

10.1 Grid and Population

- GRID_W, GRID_H:
 - affects map bounds, spawn space, render dimensions, planning space.
- NUM_AGENTS:
 - affects robot count, state vector size, planning complexity.
- NUM_SHELVES:
 - affects shelf count, state vector size, congestion, assignment.
- GOALS:
 - delivery destinations and shaping targets.

10.2 Planning and Congestion Control

- PLAN_HORIZON:
 - max time depth for space-time A*.
- ASTAR_MAX_NODES:
 - expansion cap per A* search.
- IDLE_LIMIT:
 - assignment release threshold for idle robots.
- RESERVATION_WINDOW:
 - how far ahead positions/edges are reserved.
- UNPLANNED_HOLD_STEPS:
 - conservative occupancy hold for not-yet-planned robots.
- ESCAPE_IDLE_STEPS:
 - threshold for escape fallback when pathing stalls.

10.3 Verification and Refinement

- MIN_SEPARATION:
 - safety threshold for minimum pairwise Manhattan distance.
- VERIFY_HORIZON, VERIFY_TRIALS:
 - bounded verification budget.
- REFINE_ITERATIONS:
 - max verify-refine rounds.
- REFINE_MAX_CONSTRAINTS:
 - per-iteration cap on added constraints.

10.4 Rendering

- CELL_SIZE:
 - visual scale of grid cells.
- RENDER_FPS:
 - frame pacing in environment render loop.
- colors:
 - visual style only.

11. YAML Schema (Practical)

Typical structure:

```
grid:
  width: 16
  height: 16
  cell_size: 80

agents:
  num_agents: 6
  num_shelves: 8
  goals:
    - [7, 14]
    - [8, 14]

planning:
  horizon: 30
  astar_max_nodes: 3500
  idle_limit: 4
  reservation_window: 8
  unplanned_hold_steps: 2
  escape_idle_steps: 6

render:
  fps: 2

verification:
  min_separation: 1
  horizon: 30
  trials: 20

refinement:
  iterations: 2
  max_constraints: 100
```

Only keys recognized in `load_from_yaml` are applied.

12. Precedence Rules Across the Stack

Final runtime value precedence:

1. defaults declared in `config.py`
2. overrides loaded from YAML via `cfg.load_from_yaml(...)`
3. CLI values in `main.py`:
 - explicit values (e.g., `--plan-horizon`, `--cell-size`) override both
 - optional args left `None` receive config-derived values via `_apply_defaults`.

Example:

- `PLAN_HORIZON=30` default, YAML sets `40`, CLI passes `--plan-horizon 20`:
 - final used value is `20`.

13. Failure Handling and Robustness

`load_from_yaml` failure modes and behavior:

1. Missing file:
 - logs debug, keeps defaults.
2. Missing PyYAML:
 - warning, keeps defaults.
3. Invalid root structure:
 - warning, keeps defaults.
4. Invalid value types:
 - `int(...)` conversions can raise exceptions.
 - caller (`main.py`) wraps load call in try/except and logs warning.

So configuration load errors are non-fatal to application startup.

14. Cross-Module Consumption Map

Major consumers:

- `env.py`:
 - grid/population/goals/colors/fps/cell size.
- `pathfinding.py`:
 - A* and planner tuning parameters.
- `main.py`:
 - default values for CLI optional verification/planning/refinement args.
- verification/refinement modules:
 - indirectly via args seeded from config.

15. Determinism and Experiment Reproducibility

Configuration contributes to reproducibility by fixing:

- grid dimensions,
- entity counts,
- goal geometry,
- planning and verification budgets.

Combined with seed control in `main.py`, this supports repeatable experiments.

16. Tuning Guidance (Engineering)

For heavy congestion:

- increase `PLAN_HORIZON` moderately,
- increase `ASTAR_MAX_NODES` carefully (runtime cost),
- tune `RESERVATION_WINDOW` and `UNPLANNED_HOLD_STEPS`.

For stricter safety checks:

- increase `VERIFY_TRIALS` and `VERIFY_HORIZON`,
- raise `MIN_SEPARATION` if domain requires larger clearance.

For faster visual playback:

- increase `RENDER_FPS`.

17. Full Symbol Coverage Checklist

All symbols in `config.py` covered:

- logger setup
- color constants
- environment/planning/verification/refinement constants
- `state_size`
- `STATE_SIZE`
- `load_from_yaml`

18. Chapter Summary

`config.py` is the central parameter authority for the project.

Its typed defaults, optional YAML overrides, and CLI layering enable one code-base to run in multiple regimes: interactive demo, batch simulation, evaluation, and verification-refinement experiments.

Correct interpretation of these parameters is essential for both performance and safety claims.